WMO First Mile Data Collection Guide

World Meteorological Organization

Date: 2026-06-15

Version: 0.1

Document status: DRAFT

Document location: https://wmo-im.github.io/first-mile-guide/first-mile-guide/first-mile-guide-DRAFT.html

Standing Committee on Information Management and Technology (SC-IMT)[1]

Commission for Observation, Infrastructure and Information Systems (INFCOM)[2]

Table of Contents

Chapter 1. Scope

[1] https://community.wmo.int/governance/commission-membership/commission-observation-infrastructures-and-information-systems-infcom/commission-infrastructure-officers/infcom-management-group/standing-committee-information-management-and-technology-sc-imt

[2] https://community.wmo.int/governance/commission-membership/infcom

Chapter 2. Conformance

2.1. Conformance Classes

2.1.1. Requirements Class: Core Data Model

2.1.2. Requirements Class: Standard Name Binding

2.1.3. Requirements Class: MQTT Protocol Binding

2.1.4. Requirements Class: Node

2.1.5. Requirements Class: Host

Chapter 3. Normative references

Chapter 4. Terms, definitions and abbreviations

4.1. Terms and definitions

4.1.1. First mile

4.1.2. Node device

4.1.3. Observer device

4.1.4. Observation

4.1.5. Parameter definition

4.1.6. Cell method

4.1.7. Cell period

4.1.8. Empty value

4.1.9. Standard name

4.1.10. Namespace

4.2. Abbreviations

- CF
- HMEI
- INFCOM
- MQTT
- protobuf
- QoS
- SC-IMT
- WMO

Chapter 5. Conventions

5.1. Schema notation (Protocol Buffers v3)

5.2. Requirements, recommendations and permissions

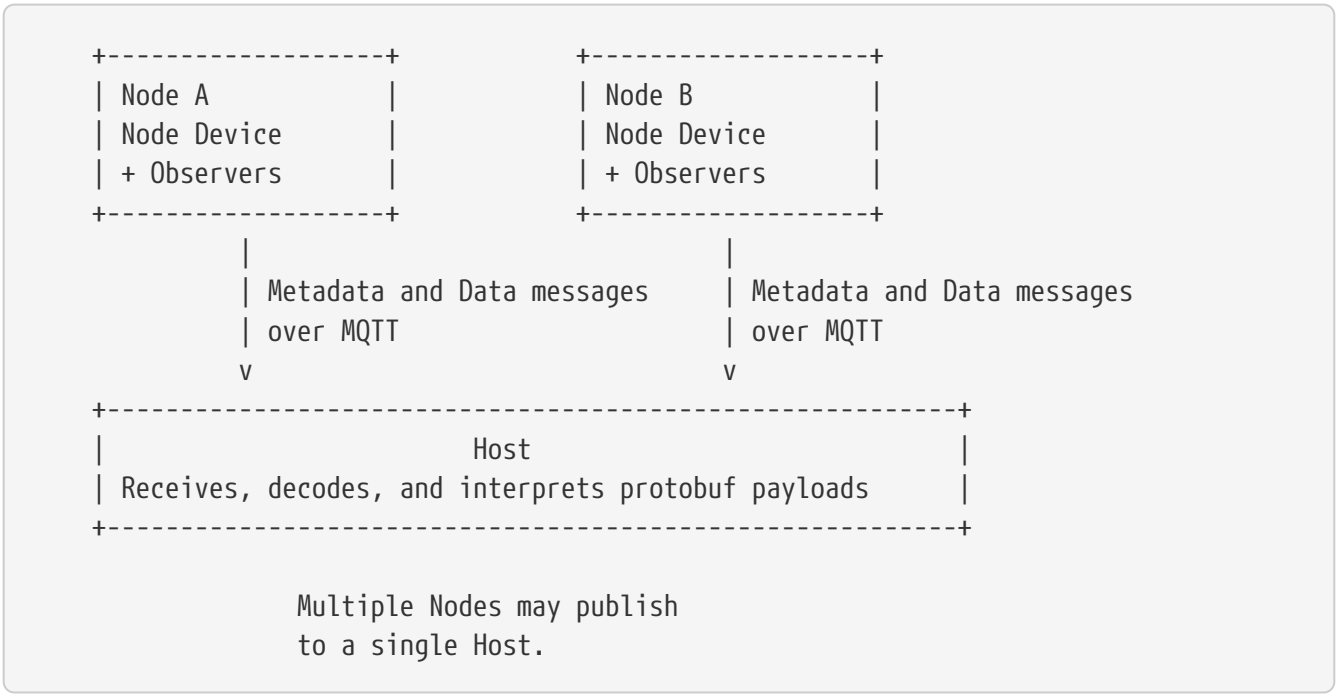
5.3. Namespace and identifier conventions

Chapter 6. Overview (informative)

This clause provides a high-level view of the First-Mile Data Collection Guide. Communication is strictly unidirectional, with the Node transmitting protobuf payloads over MQTT to the Host. The two top-level payloads are the `Data` message, which carries Observation content, and the `Metadata` message, which carries the descriptive context needed to interpret those observations.

6.1. System architecture

The Node represents the remote station or logger through a Node Device and may describe attached sensors as Observer Device entries. The `Metadata` message carries the Node Device, Observer Device, Parameter Definition, and namespace information, while the `Data` message carries one or more Observation instances that refer to those Parameter Definition identifiers. Detailed message structure is described in Clause 7, standard name binding in Clause 8, and MQTT protocol binding in Clause 9.



6.2. Message flow

In typical operation, the `Metadata` message is sent when the Node is initialized and whenever the station configuration changes, and the `Data` message is sent as observations become available. This separates relatively stable descriptive information from frequently changing measurement values and supports both individual and batched Observation delivery. Where a measurement is unavailable, the value is represented explicitly using `emptyValue` rather than being inferred from zero or omission.

6.3. Conformance class summary

Conformance class	Summary
Core Data Model	Covers the protobuf structure and semantics of the `Data` message, `Metadata` message, Observation content, devices, Parameter Definition content, and explicit missing values.
Standard Name Binding	Covers how Parameter Definition entries associate parameters with controlled standard-name vocabularies and namespaces.
MQTT Protocol Binding	Covers how protobuf payloads are conveyed over MQTT in the unidirectional exchange between Node and Host.
Node	Covers the sender-side behavior for constructing and publishing `Metadata` message and `Data` message payloads.
Host	Covers the receiver-side behavior for accepting, decoding, and interpreting payloads received from the Node.

Chapter 7. First-Mile MQTT protocol binding

- The MQTT protocol [1] is to be used for all firstmile data management workflows (publication and subscription).
- MQTT v5.0 is the chosen protocols for the publication of and subscription
- To improve the overall level of security of firstmile systems, the secure version of the MQTT protocol is preferred. If used, the certificate must be valid.
- The standard Transmission Control Protocol (TCP) ports to be used are 8883 for Secure MQTT (MQTTS) and 443 for Secure Web Socket (WSS).

7.1. First-Mile Topic Hierarchy

The First-Mile Topic Hierarchy (FMTH) provides a mechanism for systems and operators to publish and subscribe to data or metadata notifications originating from first-mile compliant environments. It is used by first-mile brokers, nodes and hubs platform.

The normative provisions in the FMTH are denoted by the base Uniform Resource Identifier (URI) `http://wis.wmo.int/spec/fmth/1` and requirements are denoted by partial URIs which are relative to this base. Topics, values and examples are represented with `shaded text`.

7.1.1. 1. Requirements Class Core

7.1.1.1. 1.1 Overview

URI	http://wis.wmo.int/spec/fmth/1/req/core
Target type	Topic classification
Dependency	MQTT v5.0
Dependency	MQTT v3.1.1
Preconditions	Topics conform to the Topic Name requirements of MQTT

The FMTH is composed of five fixed levels and one notification-type level.

All levels are mandatory. The representation is encoded as a simple text string of values at each topic level, separated by a slash (/).

Examples:

```
``firstmile/a/my-group/station-001/data``
``firstmile/a/my-group/station-001/metadata``
``firstmile/a/relief-ops/camp-alpha/data``
``firstmile/a/relief-ops/camp-alpha/metadata``
```

FMTH topic levels provides an overview of the topic levels.

Table 1. FMTH topic levels

Level	Name	Description
1	`channel`	Fixed value of `firstmile`, identifying this as a first-mile topic.
2	`version`	Alphabetical version of the topic hierarchy, currently: `a`.
3	`group-id`	Identifier for the group or organisational entity responsible for one or more first-mile nodes. It is a lowercase alphanumeric string; hyphens (`-`) may be used to separate words. The value is locally defined and shall be unique within the scope of the broker.
4	`node-id`	Identifier for the individual first-mile node (e.g. a gateway, an AWS,...). It is a lowercase alphanumeric string; hyphens (`-`) may be used to separate words. The value shall be unique within the `group-id` namespace.
5	`notification-type`	The type of notification: either `data` or `metadata`.

7.1.1.2. 1.2 Publishing

Data and metadata notifications shall be published to the exact five-level topic structure defined by the FMTH.

Examples

```
firstmile/a/my-group/station-001/data
firstmile/a/my-group/station-001/metadata
```

Requirement 1	`/req/core/publishing`	
A	A notification SHALL be published with a topic hierarchy that conforms to the FMTH structure.	
B	A `data` notification SHALL be published to a topic using the value `data` at level 5.	
C	A `metadata` notification SHALL be published to a topic using the value `metadata` at level 5.	

7.1.1.3. 1.3 Management

The topic levels are managed in a consistent manner to maintain stability and interoperability.

Requirement 2	`/req/core/management`	
A	Levels 1 and 2 (`firstmile` and `a`) SHALL be fixed and SHALL NOT be modified.	
B	The value of `group-id` (level 3) SHALL be determined and maintained by the operator of the first-mile broker.	
C	The value of `node-id` (level 4) SHALL be determined and maintained by the operator of the first-mile broker.	
D	The value of `notification-type` (level 5) SHALL be limited to `data` or `metadata`.	

7.1.1.4. 1.4 Versioning

The topic hierarchy version supports change management and transition for operators and consuming systems.

Requirement 3	`/req/core/versioning`	
A	A minor update to the FMTH SHALL NOT result in a change to the version level. Adding values to the level 5 of the FMTH (for example, adding a new notification type) is an example of a minor update that would not require a version change.	
B	A major update to the FMTH SHALL result in a change to the version level (for example, `a` becomes `b`).	
C	Removal of a topic level SHALL result in a major version update.	

Requirement 3	`/req/core/versioning`	
D	Renaming of a topic level SHALL result in a major version update.	
E	A change in the structure of the topic hierarchy SHALL result in a major version update.	

7.1.1.5. 1.5 Conventions

All levels of the topic hierarchy are defined in a consistent manner to support a normalised and predictable structure.

Requirement 4	`/req/core/conventions`	
A	Topic level definitions SHALL be lowercase.	
B	Topic level definitions SHALL be encoded using only ASCII alphanumeric characters and hyphens (`-`).	
C	Topic level definitions SHALL NOT utilise dots (`.`).	
D	Topic level definitions SHALL NOT utilise underscores (`_`).	
E	Topic level definitions SHALL utilise hyphens (`-`) to separate words (such as `my-group`).	

7.1.1.6. 1.6 Group and Node Identification

The `group-id` (level 3) identifies an organisational entity — such as an agency, project, or operational cluster — that is responsible for one or more first-mile nodes. The `node-id` (level 4) identifies a specific first-mile node within the group.

Together, `group-id` and `node-id` form the address of a first-mile data source within the FMTH namespace.

Requirement 5	`/req/core/group-id`	
A	The `group-id` SHALL consist only of lowercase ASCII alphanumeric characters and hyphens (`-`).	
B	The `group-id` SHALL NOT begin or end with a hyphen.	
C	The `group-id` SHALL NOT be empty.	

Requirement 6	`/req/core/node-id`	
A	The `node-id` SHALL consist only of lowercase ASCII alphanumeric characters and hyphens (`-`).	
B	The `node-id` SHALL NOT begin or end with a hyphen.	
C	The `node-id` SHALL NOT be empty.	
D	The `node-id` SHALL be unique within the namespace of its associated `group-id`.	

7.2. First-Mile MQTT protocol provisions

Considering the anticipated relatively low level of traffic, compare to typical MQTT situation, and the critical importance of the data and metadata exchanges in first-mile context, the following additional provisions apply to the publication of any message

7.2.1. 1. Requirements Class Core

Requirement 7	`/req/core/mqtt`	
A	Data and Metadata notifications SHALL be published using the QoS level 2 of MQTT to ensure that they are delivered and received exactly once.	
B	The connection of both the hub and the node to the broker SHALL set the **Receive Maximum** = **1** when establishing the MQTT session.	

Considering the solution used to exchange data, it is important to ensure that metadata messages are always available at the hub before any data is published. This is to ensure that consuming systems can access the necessary metadata to interpret the data correctly.

Requirement 8	`/req/core/metadata-first`	
A	Metadata notifications SHALL be published before any data notifications for the same `node-id` are published. This ensures that consuming systems have access to the necessary metadata to interpret the data correctly.	
B	Metadata notifications SHALL be published with the retain flag to true.	
C	Metadata notifications SHALL be published to the FMTH before any data notifications for the same `node-id` are published. This ensures that consuming systems have access to the necessary metadata to interpret the data correctly.	

Requirement 8	`/req/core/metadata-first`	
D	Metadata notifications SHALL published regularly, at least once every 24 hours and immediately after any change in the node configuration that would affect the metadata content.	

[1] See MQTT Specifications: https://mqtt.org/mqtt-specification/.

Chapter 8. Data model

8.1. Overview

The data model defines the structure and organization of information transmitted from the Node to the Host using Protocol Buffers (Protobuf) as the Data Format. The data model is formally specified in the protobuf schema `firstmile.proto`.

The data model supports two primary use cases:

1. **Transmission of observation data** - The Node transmits measurement values collected from sensors
2. **Transmission of metadata** - The Node transmits information about the monitoring site, devices, and parameter definitions

These use cases are supported by two top-level message types: the `Data` message containing observations, and the `Metadata` message containing configuration and contextual information. Both messages are defined in a single protobuf schema and transmitted as independent payloads over MQTT. The `Metadata` message provides the necessary context for interpreting the `Data` message, including descriptions of devices and parameters.

8.1.1. Communication pattern

The communication follows a unidirectional pattern where data flows from the Node to the Host. The Node may transmit messages in the following patterns:

- **Data-only transmission** - Frequent transmission of `Data` messages containing current or batched observations
- **Metadata transmission** - Transmission of `Metadata` messages when the configuration changes or upon initialization

The Node should transmit the `Metadata` message at least once after initialization and whenever the configuration of devices or parameter definitions changes. The `Data` message may be transmitted as frequently as needed to deliver observations. The Host can only intepret data correctly if it has received the relevant `Metadata` message that defines the devices and parameters being observed. Therefore, it is recommended to transmit the `Metadata` message at least once before transmitting any `Data` messages.

8.1.2. Message hierarchy

The data model organizes information hierarchically:

- **Messages** - Top-level containers (`Data` and `Metadata`) that are transmitted as independent payloads
- **Devices** - Descriptions of physical equipment including the Node Device (node) and Observer Devices (sensors)
- **Observations** - Measurement data points consisting of a parameter identifier, timestamp, and

one or more values

- **Parameter definitions** - Metadata describing what is being measured, including units, aggregation methods, and standard name mappings. Parameter definitions also describe how observations are structured in `Data` messages.
- **Values** - The actual measurement data, supporting multiple numeric types, strings, booleans, and explicit representation of missing data

Each `Observation` references a `ParameterDefinition` by identifier. Each `ParameterDefinition` contains one or more `Parameter` specifications that describe the measured variable, its unit, temporal aggregation method, and which device performs the measurement.

8.1.3. Design principles

The data model is designed according to the following principles:

- **Efficiency** - Optimized for bandwidth-constrained environments but focusing on IP communication rather than extremely low-level binary formats
- **Extensibility** - Support for multiple value types and flexible parameter definitions
- **Self-description** - Metadata provides complete context for interpreting observations
- **Explicit missing values** - Clear distinction between unavailable data and zero/false measurements
- **Batch transmission** - Multiple observations may be included in a single `Data` message
- **Standard name interoperability** - Mapping to external naming conventions through namespace definitions

8.2. Messages

8.2.1. Metadata message

The `Metadata` message provides configuration and contextual information about the Node, including descriptions of devices and the parameters they measure. This message shall be transmitted at least once after Node initialization and whenever the configuration changes.

The `Metadata` message contains the following fields:

Field	Type	Cardinality	Description
node	Node	required	Description of the Node device (datalogger or station)
observers	ObserverDevice	repeated	Descriptions of individual sensor devices connected to the node
parameterDefinitions	ParameterDefinition	repeated	Definitions of the parameters used in observations

Field	Type	Cardinality	Description
namespaces	map<string, string>	optional	Mapping of namespace prefixes to their corresponding URIs for standard name resolution

The `namespaces` field provides the mapping between namespace keys used in `Parameter.standardNames` and their full URIs or identifiers. For example, mapping the key `"cf"` to the URI `"https://vocab.nerc.ac.uk/collection/P07/current/"` enables the use of Climate and Forecast (CF) convention standard names.

8.2.2. Data message

The `Data` message is the primary container for transmitting observation data from the Node to the Host. This message may be transmitted as frequently as needed to deliver current or batched observations.

The `Data` message contains the following fields:

Field	Type	Cardinality	Description
observations	Observation	repeated	Zero or more observation measurements

A `Data` message may contain multiple `Observation` messages, enabling efficient batch transmission of measurement data. Each observation is independent and contains its own timestamp and parameter reference.

NOTE The Node may transmit `Data` messages containing observations that reference parameter definitions not yet received by the Host. In this case, the Host shall buffer or queue such observations until the corresponding `Metadata` message is received.

8.3. Devices

8.3.1. Node device

The `Node` message describes the Node device itself, typically a datalogger or observation station.

The `Node` message contains the following fields:

Field	Type	Cardinality	Description
name	string	required	Name or model designation of the node device
location	Location	optional	Geographic location of the node device
url	string	optional	URI reference to additional metadata about the device

Field	Type	Cardinality	Description
serialNumber	string	optional	Manufacturer's serial number of the device
firmwareVersion	string	optional	Firmware version of the device, typically in format such as "5.1" or "2.7.1-alpha"

The `name` field should contain a human-readable identification of the device, such as its model name or type designation.

8.3.2. Observer device

The `ObserverDevice` message describes an individual sensor or measurement device connected to the Node Device.

The `ObserverDevice` message contains the following fields:

Field	Type	Cardinality	Description
id	uint32	required	Unique identifier for this observer within the context of this node
name	string	required	Name or model designation of the observer device
location	Location	optional	Geographic location of the observer device
url	string	optional	URI reference to additional metadata about the device
serialNumber	string	optional	Manufacturer's serial number of the device
firmwareVersion	string	optional	Firmware version of the device

The `id` field shall be unique among all observers associated with a single node device. This identifier is used by the `DeviceRef` message to associate parameters with specific observer devices.

IMPORTANT	Observer device identifiers shall remain stable across Node restarts and metadata updates to ensure consistent parameter-to-device associations.

8.3.3. Location and reference surfaces

The `Location` message specifies the geographic position of a device (either node or observer).

The `Location` message contains the following fields:

Field	Type	Cardinality	Description
latitude	double	optional	Latitude in decimal degrees, range -90 to +90
longitude	double	optional	Longitude in decimal degrees, range -180 to +180

Field	Type	Cardinality	Description
heightMeter	double	required	Height in meters above the reference surface
referenceSurface	ReferenceSurface	required	The reference surface for the height measurement

The `latitude` and `longitude` fields may be omitted when only the local height of a device is known or relevant. The `heightMeter` field shall always be provided together with a `referenceSurface` specification.

The `ReferenceSurface` enumeration defines the following values:

Value	Description
REFERENCE_SURFACE_UNSPECIFIED	Reserved default value. Shall not be explicitly set by Nodes.
MSL	Mean Sea Level - the average height of the ocean's surface
GEOID	Geoid - an equipotential surface of the Earth's gravity field
GL	Ground Level - the local ground or surface level at the site
REFERENCE_ELLIPSOID	Reference Ellipsoid - a mathematically defined surface approximating the Earth's shape (e.g., WGS84)
PRESSURE_1000_HPA	The 1000 hPa pressure level in the atmosphere

NOTE The choice of reference surface is determined by the user based on the measurement context and requirements. For example, meteorological observations of air temperature typically use height above ground level (GL), while atmospheric pressure observations may reference mean sea level (MSL).

8.4. Observations and values

8.4.1. Observation

The `Observation` message represents a single measurement data point collected at a specific time.

The `Observation` message contains the following fields:

Field	Type	Cardinality	Description
parameterDefinitionId	uint32	required	Identifier referencing a `ParameterDefinition` in the `Metadata` message
time	google.protobuf.Timestamp	required	The timestamp when the observation was made, in UTC
values	Value	repeated	One or more measurement values

The `parameterDefinitionId` field shall reference a `ParameterDefinition.id` from the `Metadata` message. The number and type of values in the `values` field shall correspond to the number of parameters defined in the referenced `ParameterDefinition`.

The `time` field uses the standard `google.protobuf.Timestamp` format, representing time as seconds and nanoseconds since the Unix epoch (1970-01-01T00:00:00Z).

NOTE

An observation may contain multiple values when the referenced `ParameterDefinition` contains multiple parameters. This approach enables grouping values that are measured at the same timestamp, requiring the timestamp to be transmitted only once for multiple values. This reduces payload size, which is particularly beneficial in bandwidth-constrained environments.

For example, a parameter definition for "Internal Parameters" might include both supply voltage and internal temperature, requiring two values per observation but sharing a single timestamp.

8.4.2. Value types

The `Value` message is a polymorphic container that can represent measurement data of different types. The message uses a `oneof` field, meaning exactly one of the following value types shall be set:

Field	Type	Description
floatValue	float	32-bit floating point number
doubleValue	double	64-bit floating point number
intValue	int32	32-bit signed integer
unsignedIntValue	uint32	32-bit unsigned integer
int64Value	int64	64-bit signed integer
unsignedInt64Value	uint64	64-bit unsigned integer
stringValue	string	Text string value
boolValue	bool	Boolean true/false value
emptyValue	google.protobuf.Empty	Indicates missing or unavailable data

The choice of value type should be appropriate for the parameter being measured. Numeric measurements typically use `doubleValue` or `floatValue`, while categorical observations may use `stringValue`. Boolean parameters (such as alarm states) use `boolValue`.

NOTE

The `stringValue` field is intended for text data only and shall NOT be used to transmit:

- Binary data, including base64-encoded binary blobs
- Serialized objects or data structures
- Structured data formats such as JSON, XML, or similar encodings

> Binary data transmission and complex structured data are outside the scope of this standard.

8.4.3. Missing and error values

When a measurement value is unavailable due to sensor malfunction, communication error, or other causes, the value shall be represented using the `emptyValue` field of type `google.protobuf.Empty`.

IMPORTANT

Missing or unavailable measurement values shall NOT be represented as:

- Numeric zero (0)
- NaN (Not a Number)
- Omission from the `values` array
- Any other numeric sentinel value

The `emptyValue` field shall be used to explicitly and unambiguously indicate missing data.

This approach ensures clear distinction between actual measurements of zero and missing measurements, which is critical for correct data interpretation.

When an `Observation` references a `ParameterDefinition` containing multiple parameters, the values in the `values` array shall be provided in the same order as the parameters are defined in the `ParameterDefinition.parameters` field. It is not permitted to omit a value from the middle of the array, as this would cause misalignment with the parameter definitions. If a value is unavailable, the `emptyValue` field shall be used at that position to maintain correct alignment.

8.5. Parameter definitions

A `ParameterDefinition` groups one or more related measurement parameters that share common context. Each parameter definition has a unique identifier that is referenced by observations. Parameters are typically grouped when they are measured at the same timestamp, allowing multiple values to share a single timestamp in observations and thus reducing payload size.

The `ParameterDefinition` message contains the following fields:

Field	Type	Cardinality	Description
id	uint32	required	Unique identifier for this parameter definition
description	string	required	Human-readable description of this parameter definition
parameters	Parameter	repeated	One or more parameter specifications

The `id` field shall be unique within the scope of a single `Metadata` message. This identifier is used by the `Observation.parameterDefinitionId` field to associate observations with their parameter

specifications.

The `parameters` field contains one or more `Parameter` messages describing individual measurable variables. When multiple parameters are grouped in a single definition, each observation referencing this definition shall provide values in the same order as the parameters are defined.

8.5.1. Parameter

The `Parameter` message specifies a single measurable variable, including its name, unit of measurement, aggregation method, and associated device.

The `Parameter` message contains the following fields:

Field	Type	Cardinality	Description
longName	string	required	Descriptive name of the parameter (e.g., "Air Temperature", "Wind Speed")
unit	string	required	Unit of measurement (e.g., "°C", "m/s", "hPa")
standardNames	map<string, string>	optional	Mapping of namespace prefixes to standard names in external naming conventions
device	DeviceRef	required	Reference to the device (node or observer) that measures this parameter
cellMethod	CellMethod	required	Statistical or aggregation method applied to the measurement
cellPeriodSeconds	uint32	required	Period of aggregation in seconds (0 for instantaneous point measurements)

The `standardNames` field enables mapping to external naming conventions such as Climate and Forecast (CF) standard names. This allows parameters to be referenced by different standard parameter types according to the target application of the device using the protocol. The Node is not bound to a specific standard but may include mappings to multiple standards as appropriate for the intended use case. The keys in this map shall correspond to namespace prefixes defined in `Metadata.namespaces`.

The `device` field specifies which device (either the node or a specific observer) performs this measurement.

8.5.2. Cell method

The `cellMethod` field specifies how measurement data is aggregated or processed over the cell period. The `CellMethod` enumeration defines the following values:

Value	Description
CELL_METHOD_UNSPECIFIED	Reserved default value. Shall not be explicitly set by Nodes.

Value	Description
POINT	Instantaneous value at a specific point in time (no aggregation)
SUM	Sum of values over the cell period
MAXIMUM	Maximum value observed during the cell period
MINIMUM	Minimum value observed during the cell period
MEAN	Mean (average) value over the cell period
MEDIAN	Median value over the cell period
MODE	Most frequently occurring value
VARIANCE	Statistical variance within the cell period
STANDARD_DEVIATION	Standard deviation within the cell period
MAXIMUM_ABSOLUTE_VALUE	Maximum of absolute values
MINIMUM_ABSOLUTE_VALUE	Minimum of absolute values
MEAN_ABSOLUTE_VALUE	Mean of absolute values
MEAN_OF_UPPER_DECILE	Mean of the upper tenth of the value distribution
MID_RANGE	Average of maximum and minimum values
RANGE	Absolute difference between maximum and minimum values
ROOT_MEAN_SQUARE	Root mean square (RMS) value
SUM_OF_SQUARES	Sum of squared values

NOTE When `cellMethod` is set to `POINT`, the measurement represents an instantaneous value and `cellPeriodSeconds` shall be 0. For all other cell methods, `cellPeriodSeconds` shall specify the duration over which the aggregation is performed.

8.5.3. Cell period

The `cellPeriodSeconds` field specifies the time period in seconds over which the cell method is applied.

- When `cellMethod` is `POINT`, the value shall be 0, indicating an instantaneous measurement.
- For all other cell methods, the value shall be greater than 0, indicating the duration of the aggregation period.

For example: * A 30-second mean temperature would have `cellMethod` = `MEAN` and `cellPeriodSeconds` = 30 * An instantaneous wind direction would have `cellMethod` = `POINT` and `cellPeriodSeconds` = 0 * A 5-minute maximum wind gust would have `cellMethod` = `MAXIMUM` and `cellPeriodSeconds` = 300

8.5.4. Device reference

The `DeviceRef` message associates a parameter with the device that measures it. This message uses a `oneof` field to reference either the node device or a specific observer device.

The `DeviceRef` message contains exactly one of the following fields:

Field	Type	Description
node	google.protobuf.Empty	References the node device; the presence of this field (even though empty) indicates the parameter is measured by the node
observerId	uint32	References an observer device by its `ObserverDevice.id` value

The `oneof` constraint ensures that exactly one of these fields is set, unambiguously identifying which device performs the measurement.

When the `node` field is present, the parameter is measured by the Node Device itself (such as internal voltage or temperature of a datalogger). When the `observerId` field is set, it shall match the `id` of an `ObserverDevice` defined in the `Metadata` message.

Chapter 9. Standard name binding

9.1. Overview

Standard name binding enables `Parameter` definitions within a `Metadata` message to be associated with controlled vocabulary entries from external naming conventions. This allows observation data to be unambiguously interpreted by Hubs and downstream systems without dependence on the `Parameter.longName` field, which is a human-readable label and not guaranteed to be unique or machine-interpretable.

The binding mechanism uses two related fields in the data model:

- `Metadata.namespaces` — declares the naming convention vocabularies in use, associating a short namespace key with a URI that identifies the vocabulary
- `Parameter.standardNames` — associates each namespace key with the corresponding standard name for the parameter within that vocabulary

Standard name binding follows a three-level hierarchy of vocabularies, described in Standard name hierarchy. The use of standard names is optional but recommended where machine-to-machine interoperability is required. A Node may bind a single parameter to standard names from multiple vocabularies simultaneously.

9.2. Namespace declaration

The `Metadata.namespaces` field declares the naming convention vocabularies used by the Node. It is a map from namespace key to namespace URI.

Map entry	Type	Description
key	string	A short identifier used to reference this namespace in `Parameter.standardNames` entries (e.g., `"cf"`, `"wmo"`)
value	string	A URI that uniquely and durably identifies the vocabulary (e.g., `"https://vocab.nerc.ac.uk/collection/P07/current/"`)

Each namespace key shall be a non-empty string containing only alphanumeric characters and underscores. The namespace URI shall be a resolvable, publicly accessible URI that identifies the vocabulary in a dereferenceable way.

A `Metadata` message may declare zero or more namespace entries. When no standard names are used, the `namespaces` map may be empty or omitted.

9.3. Standard name assignment

The `Parameter.standardNames` field maps namespace keys to the standard name for that parameter within the corresponding vocabulary. It is a map from namespace key to standard name value.

Map entry	Type	Description
key	string	A namespace key that shall match a key declared in `Metadata.namespaces`
value	string	The standard name for this parameter in the vocabulary identified by the namespace key (e.g., `"air_temperature"`)

Each key in `Parameter.standardNames` shall match a key declared in `Metadata.namespaces`. A Node shall not include a standard name entry for a namespace that is not declared in the corresponding `Metadata` message.

A parameter may carry zero, one, or multiple standard name entries. When multiple entries are present, each entry associates the parameter with the equivalent standard name in a different vocabulary.

NOTE Standard name binding is applied at the `Parameter` level, not the `ParameterDefinition` level. When a `ParameterDefinition` contains multiple `Parameter` entries, each parameter independently carries its own standard name bindings appropriate for the variable it represents.

9.4. Standard name hierarchy

Standard names shall be drawn from the following three-level hierarchy, in order of precedence. A higher-level vocabulary shall be used wherever it provides a suitable standard name for the parameter in question.

9.4.1. Level 1: CF Metadata Conventions

The Climate and Forecast (CF) Metadata Conventions define the primary vocabulary for standard names in this standard. CF standard names cover the majority of meteorological, hydrological, and oceanographic parameters relevant to first-mile observations. The authoritative list of CF standard names is maintained by the CF Conventions community at https://cfconventions.org/Data/cf-standard-names/current/build/cf-standard-name-table.html.

Where a CF standard name exists for a parameter, the Node shall use it in preference to any lower-level vocabulary entry.

The CF namespace shall be declared using the reserved key `"cf"`. The value shall be a URI that resolves to a machine-readable vocabulary server endpoint publishing the CF standard name table.

The CF Conventions community does not currently operate a vocabulary server. Until an official CF vocabulary server endpoint is available, the NERC Vocabulary Server mirror of the CF standard name table (collection P07) should be used:

```
"namespaces": {
  "cf": "https://vocab.nerc.ac.uk/collection/P07/current/"
}
```

NOTE	If the CF Conventions community publishes an official vocabulary server endpoint in the future, that URI should be used in preference to the NERC mirror. Hubs should be prepared to recognise URIs from recognised CF standard name vocabulary server mirrors as equivalent for the purposes of name resolution.

CF standard names are lower-case strings with words separated by underscores (e.g., `"air_temperature"`, `"wind_speed"`, `"relative_humidity"`). The CF standard name table at cfconventions.org is the authoritative reference for name validity and definitions, regardless of which vocabulary server URI is used as the namespace identifier.

9.4.2. Level 2: WMO First Mile parameter registry

Parameters that are not covered by the CF standard name vocabulary but are common to first-mile observation deployments are governed by a centrally managed registry maintained by WMO and hosted at `https://codes.wmo.int`.

Where a parameter is not covered by CF but is included in the WMO First Mile parameter registry, the Node shall use the registry entry in preference to any locally managed vocabulary.

The WMO First Mile registry namespace shall be declared using the reserved key `"wmo"` with the URI of the registry collection.

NOTE	Parameters that may require Level 2 registry entries include quantities typically measured by data loggers but absent from CF, such as supply voltage, internal device temperature, and station-level pressure (QFF). The registry is managed separately from this standard; refer to `https://codes.wmo.int` for the current list of registered entries.

9.4.3. Level 3: Locally managed vocabularies

Where a parameter is not covered by either the CF standard name vocabulary (Level 1: CF Metadata Conventions) or the WMO First Mile parameter registry (Level 2: WMO First Mile parameter registry), a Node may use a locally managed vocabulary.

Locally managed vocabularies shall conform to the FAIR Guiding Principles for scientific data management (Findable, Accessible, Interoperable, Reusable). In particular:

- The vocabulary shall be findable via a persistent, unique identifier (the namespace URI)
- The vocabulary shall be accessible at that URI to all parties participating in the data exchange
- The vocabulary shall use an open, standardised representation (e.g., SKOS/RDF)
- Entries shall be stable; their identifiers and meanings shall remain unchanged once published

NOTE	A conformant Hub shall be capable of resolving namespace URIs as configured for the deployment, including URIs accessible only within a private or organisational network. A conformant Node shall use namespace URIs that are resolvable by the Hub within the operational context of the deployment.

Local namespace entries may be included alongside Level 1 or Level 2 standard names. A Hub shall be able to identify and interpret any parameter using the standard names alone.

IMPORTANT	Where a standard name exists at Level 1 (CF) or Level 2 (WMO registry), that standard name shall be included in `Parameter.standardNames`. Any locally managed entries for the same parameter are supplementary to, not a replacement for, the applicable standard name.

9.5. Informative example

The following example shows a `Metadata` message extract demonstrating standard name binding across multiple vocabulary levels for a meteorological and hydrological station.

```
{
  "parameterDefinitions": [
    {
      "id": 1,
      "description": "Surface meteorological observations",
      "parameters": [
        {
          "longName": "Air Temperature",
          "unit": "°C",
          "standardNames": { "cf": "air_temperature" },
          "device": { "observerId": 1 },
          "cellMethod": "MEAN",
          "cellPeriodSeconds": 60
        },
        {
          "longName": "Wind Speed",
          "unit": "m/s",
          "standardNames": { "cf": "wind_speed" },
          "device": { "observerId": 2 },
          "cellMethod": "MEAN",
          "cellPeriodSeconds": 600
        }
      ]
    },
    {
      "id": 2,
      "description": "Station power and diagnostics",
      "parameters": [
        {
          "longName": "Supply Voltage",
          "unit": "V",
          "standardNames": { "wmo": "supply_voltage" },
          "device": { "host": {} },
          "cellMethod": "POINT",
          "cellPeriodSeconds": 0
        }
```

```
      ]
    }
  ],
  "namespaces": {
    "cf": "https://vocab.nerc.ac.uk/collection/P07/current/",
    "wmo": "https://codes.wmo.int/first-mile/standard-names/"
  }
}
```

Chapter 10. Protocol binding

10.1. Overview

10.2. MQTT version requirements

10.3. Topic structure

10.3.1. Topic pattern

10.3.2. Level definitions and allowed values

10.4. Quality of service

10.4.1. Metadata messages

10.4.2. Data messages

Chapter 11. Data sender

11.1. Metadata publication

A key feature of the First Mile Format, is the structured inclusion of metadata. This enables

Benefit	Definition	Required in implementation
Machine Translation	Received data can be correctly assigned to observations in database/products/services without human interpretation/intervention	Standard Names
Stranger Interpretation	Siting (particularly location and height of observations) and equipment can be deduced from data, enabling assessment of observation quality without access to other information	Location, URL (for additional information)
Efficient Communication	Metadata is not included in every message (reducing bandwidth requirements), only to ensure efficient processing by the Receiver	Transmission Triggers

11.1.1. Mandatory fields

To enable evolution of the First Mile Standard, while maximising backward compatability, the Protobuf **REQUIRED** keyword is not used. However, omission of key values can render the other included information unusable as it is ambiguous as to how it is related to the other content.

Message	Field	Justification
Node	name	Enables the identification of the end node
ObserverDevice	id	Enables parameters (observation data) to be matched to the sensing/observing/measuring device
ParameterDefinition	id	Enables content from Data messages to be interpreted
Parameter	unit	Critical to hub interpretation of received data

Parameter	DeviceRef	Enables sensors/observers to be matched to their data

If machine to machine translation is expected, then the following following additional fields must be included

Message	Field	Justification
Parameter	standardNames	Enables machine to machine translation of definition, irrespective of parameter name
Parameter	CellMethod	If any data processing is done at the edge, this enables, for example, the mean and standard deviation to be differentiated (as they will have the same standardNames, units)

11.1.2. Transmission triggers

The hub is unable to process any Data Messages without a current Metadata Message.

As a minimum, a new/updated Metadata Message must be transmitted whenever the Metadata changes. This is particularly important when:

- Sensors are added or removed
- New Observations (or statistics on existing observations, for example max/min values) are provided.

To balance network transmission costs for Metadata Messages (which are typically significantly larger than Data Messages) with overhead of storing Data messages until the relevant Metadata message is received and the Data Messages can be understood, recommended practice is (re)sending of the Metadata Message:

- on power-on/restart of hub
- on a "heartbeat" schedule of at least once per day.

11.1.3. Retained messages

The Data Messages cannot be understood without their associated Metadata Message. To meet Archiving requirements [1] in most jurisdictions the Data Messages, and associated Metadata Messages must both be retained. Typically, this can be a one (Metadata Message) to many (Data Messages) relationship

[1] For Example WMO 1238 (2023) 2.5.5

Chapter 12. Data publication

The First Mile Format enables efficient communication of Data:

- the message is a compact/efficient binary format.
- only the Observation/Data values are transmitted [as the less often transmitted Metadata Message carries the information to understand the values]

12.1. Mandatory fields

Every Data message must contain at least:

parameterDefinitionId	Enables the Data to be matched to the definition from the Metadata Message
time	
(at least one) value	

12.2. Observation timestamps

The Data (Observation) Timestamp uses the google.protobuf.Timestamp structure for each observation.

```
message Timestamp {
    int64 seconds = 1; // seconds since UTC (Unix) epoch (1/1/1970)
    int32 nanos = 2; // non-negative nanosecond fractions
}
```

The structure enables date/time resolutions to nanoseconds for observations.

12.3. Device identification

The Data Message does not contain any Station/Device information in the message. Instead, the MQTT topic hierarchy NodeID (Level 4) [and possibly GroupID (Level 3)] would be used for Station/Device Identification and should be archived with the Data Message.